

Fabian 
@ffaebi

Follow ...

Creating, modifying and hosting Maps in Ruby on Rails: A rough Guide



↻ 1

♡ 5

📊 891



Note: This is a **very rough guide** to point you in the right direction, not a perfect step by step guide. I'm focusing strongly on openstreetmap (OSM). I use ruby on rails, but it should be reproducible in most web frameworks. This article has been extracted out of my own documentation and probably contains some errors.

In a nutshell, tooling that works:

- (Ruby on Rails)
- **Openstreetmap** data dump as **.osm.pbf** from geofabrik
- **postgres + postgis** as database
- **imposm3** to create import .osm.bpf into postgres



- **protomaps** to use **vector maps** instead of images
- protomaps to enable **selfhosting** of vector maps

Table of Contents

1. Introduction to **OpenStreetMap (OSM)**
2. Setting Up the Environment
3. Importing OSM Data with **imposm3**
4. Configuring the Database
5. Integrating with Ruby on Rails
6. Displaying Data on a Map using **maplibre**
7. Changing the map
8. Selfhosting the vector tiles
9. Tips and tricks

1. Introduction to OpenStreetMap

OpenStreetMap (OSM) is essentially a vast database of geographical information. It provides regular exports in **.osm.pbf** format, available for the entire world or smaller areas like individual countries. For beginners, it's recommended to start with a smaller dataset, such as Liechtenstein, which can be found on Geofabrik (<https://download.geofabrik.de/>).

2. Setting Up the Environment

To work with OSM data, you'll need:

- Ruby on Rails
- PostgreSQL with **PostGIS** extension



For PostgreSQL with PostGIS, you can use the Docker image:

```
shell
docker run --name postgis -e POSTGRES_PASSWORD=mysecretpassword -d postgis/postgis
```

Install Imposm3 following the instructions on their GitHub repository.

3. Importing OSM Data with Imposm3

Imposm3 requires a YAML file to describe which data to import. Please follow their messy github repo for details. Here's an example for importing trashbins:

```
yaml
# mapping.yaml
tables:
  trashbins:
    type: point
    columns:
      - name: osm_id
        key:
          type: id
      - name: geometry
        key:
          type: geometry
    mapping:
      amenity:
        - waste_basket
```

To import data into postgres:

```
shell
imposm import -mapping mapping.yaml -read liechtenstein-latest.osm.pbf -write
```

After creating the intermediate table, run this to overwrite the production table. That also enables you seamless data updates in the future:

```
imposm import -mapping mapping.yaml -connection "postgres://postgres:mysecretpa
```

4. Configuring the Database

Ensure your database.yml is configured to use PostGIS:

```
yaml
default: &default
  adapter: postgis
  encoding: unicode
  pool: <%= ENV.fetch("RAILS_MAX_THREADS") { 5 } %>

development:
  <<: *default
  database: map_development
```

5. Integrating with Ruby on Rails

Add these gems to your Gemfile:

```
ruby
gem "activerecord-postgis-adapter"
gem "rgeo-activerecord"
```

Create a model with a special table name for your imported data:

```
ruby
# app/models/trashbin.rb
class Trashbin < ApplicationRecord
  self.table_name = "osm_trashbins"
end
```

Now you can query the data:

```
ruby
Trashbin.first
# => #<Trashbin:0x00007ffad2435010
# id: 1,
# osm_id: -17923463,
# geometry: #<RGeo::Geos::CAIPolygonImpl:0xdf34 "POINT ((785144.9893625469 5
```

6. Displaying Data on a Map using manlibre



xml



```

<script src='https://unpkg.com/maplibre-gl@2.4.0/dist/maplibre-gl.js'></script>
<link href='https://unpkg.com/maplibre-gl@2.4.0/dist/maplibre-gl.css' rel='sty
<script src="https://unpkg.com/pmtiles@2.7.0/dist/index.js"></script>

<div id="map"></div>
<script>
  var map = new maplibregl.Map({
    container: 'map',
    style: 'https://api.protomaps.com/styles/v2/light.json?key=<YOUR_KEY>',
    center: [0, 0],
    zoom: 1
  });

  map.on('load', () => {
    map.addSource('custom-trashbins', {
      'type': 'geojson',
      'data': {
        'type': 'FeatureCollection',
        'features': [
          <%- @trashbins.each do |trashbin| -%>
          {
            'type': 'Feature',
            'geometry': {
              'type': 'Point',
              'coordinates':
            <%=
              if trashbin.geometry.is_a?(RGeo::Geos::CAPIPointImpl)
                point = trashbin.geometry
                lon = point.x * 180 / 20_037_508.34
                lat = Math.atan(Math.exp(point.y * Math::PI / 20_037_508.34))
                [lon, lat].to_json
              else
                '[0, 0]'
              end
            <%=
              },
            'properties': {
              'color': '<%= [:red, :green, :blue].sample %>'
            }
          }
          <%= ", " unless trashbin == @trashbins.last %>
          <%- end -%>
        ]
      }
    });

    map.addLayer({
      'id': 'custom-trashbins',
      'type': 'circle',
      'source': 'custom-trashbins',
      'paint': {
        'circle-radius': 6,
        'circle-color': ['get', 'color']
      }
    });
  });
</script>

```

That should hopefully give you a simple map with points.

7. Changing the Map



`https://api.protomaps.com/styles/v2/light.json?key=<YOUR_KEY>`

- The style can be updated using [maputnik](#)

8. Selfhosting the vector tiles

- You can download your own world map in the .pmtiles format [from protomaps](#).
- You can stream that gigantic .pmtiles from rails using the following Controller action called `tiles`

ruby



```
include ActionController::Live

MAX_RANGE_SIZE = 10.megabytes
def tiles
  file_path = Rails.root.join("storage/20240911.pmtiles")
  file_size = File.size(file_path)

  unless request.headers["Range"]
    render plain: "Range header is required", status: :bad_request
    return
  end

  match = request.headers["Range"].match(/bytes=(\d+)-(\d*)/)

  if match
    start = match[1].to_i
    finish = match[2].present? ? [match[2].to_i, start + MAX_RANGE_SIZE - 1]
    length = finish - start + 1

    if length > MAX_RANGE_SIZE
      render plain: "Requested range exceeds maximum allowed size", status:
      return
    end

    response.header["Content-Range"] = "bytes #{start}-#{finish}/#{file_size}"
    response.header["Content-Length"] = "#{length}"
    response.header["Accept-Ranges"] = "bytes"
    response.header["Content-Type"] = "application/vnd.pmtiles"

    send_data IO.binread(file_path, length, start), type: "application/vnd.p"
  else
    render plain: "Invalid range request", status: :requested_range_not_sati
  end
end
```

Congratulations, your map is now fully self-hosted.



- Maps are complex, **use AI** to help you.
- OSM tagging can be complex.
- This [OSM Wiki](#) page contains nearly all commonly used tags
- Start with small datasets (like Liechtenstein) for easier testing and iteration.
- Don't modify the `osm_` tables directly; they'll be overwritten on fresh imports. Instead, join your own data using the `osm_id`.
- The `imposm3` `mapping.yml` file can be tricky. Take time to get it right.
- Use Maplibre and not leaflet
- Be cautious with latitude and longitude; they're often mixed up.
- Consider using Protomaps for a fully self-hosted map experience.
- You can style your own map using [Maputnik](#).

By following this guide, you should now have a solid foundation for working with OpenStreetMap data in your Ruby on Rails application.

 Want to publish your own Article?

[Upgrade to Premium+](#)

9:00 PM · Sep 15, 2024 · **891** Views



